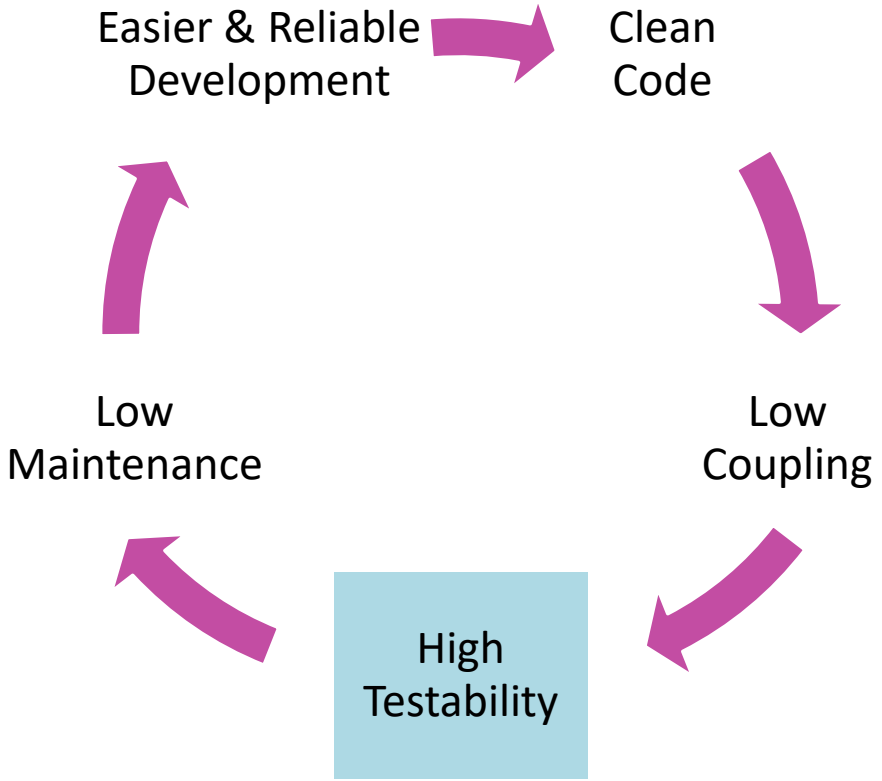
Writing Testable Code

KAREL FRAJTÁK

Motivation

Goal

- Easy testing
- Clean code
- High modularity
- High cohesion
- Low coupling
- No code smells



Unit Testing

Automated process of **validating the behavior** within an application in **isolation** from any of its dependencies

Test doubles are used to isolate the behavior of components.

Isolated testing helps to identify the components causing a **regression** in behavior.

Often overlooked because

- Lack of knowledge – I don't know how to write testable code and tests
- Code is hard to test – This is a legacy application, and it is impossible to test it.
- There is no time for testing – Development team is rushing toward the release date and there is no time for testing of any kind.
- It's just my hobby project

Testable and Untestable Code

The testing process should start in the developer's head when writing code.

Developer should follow the guidelines for writing testable code and avoid anti-patterns, code smells and bad coding practices like

- SOLID principles violation
- Non-deterministic code (hardcoded `LocalDateTime.now()`)
- Code with side-effects
- Static (global) code
- Singletons
- The `new` operator

Code Smells

Code smell is any characteristic in the source code of a program that possibly indicates a deeper problem. Determining what is and is not a code smell is subjective, and varies by language, developer, and development methodology.

Smells are certain structures in the code that indicate violation of fundamental design principles and negatively impact design quality.

Code smells are usually not bugs, they are not technically incorrect and do not prevent the program from functioning. Instead, they indicate weaknesses in design that may slow down development or increase the risk of bugs or failures in the future.

Bad code smells can be an indicator of factors that contribute to technical debt.

Code Smells

- Duplicated Code
- Shotgun Surgery
  - A change in one class requires cascading changes in several related classes
- God Class
- Feature Envy
  - Methods that make extensive use of another class may belong in another class
- Inappropriate Intimacy
  - Classes should know as little as possible about each other
- Too Many Parameters
- God Line
- Comments
- Cyclomatic Complexity
- Dead Code
- Primitive Obsession (If data type is sufficiently complex, write a class to represent it)
- Inconsistent Names
- Conditional Complexity

And more

- https://blog.codinghorror.com/code-smells/
- http://mikamantyla.eu/BadCodeSmellsTaxonomy.html

Test Smells

Erratic Test	One or more tests are behaving erratically; sometimes they pass and sometimes they fail.
Obscure Test	It is difficult to understand the test immediately.
Fragile Test	A test fails to compile or run when the SUT is changed in ways that do not affect the part the test is exercising.
Slow Test	The tests take too long to run.
Assertion Roulette	It is hard to tell which of several assertions within the same test method caused a test failure.
Conditional Test Logic	A test contains code that may or may not be executed
Hard-to-test Code	Code is difficult to test.
Test Code Duplication	The same test code is repeated many times.
Production Bugs	We find too many bugs during formal test or in production.
Developers Not Writing Tests	Developers aren't writing automated tests.

AAA & Fluent Assertions

Test suites

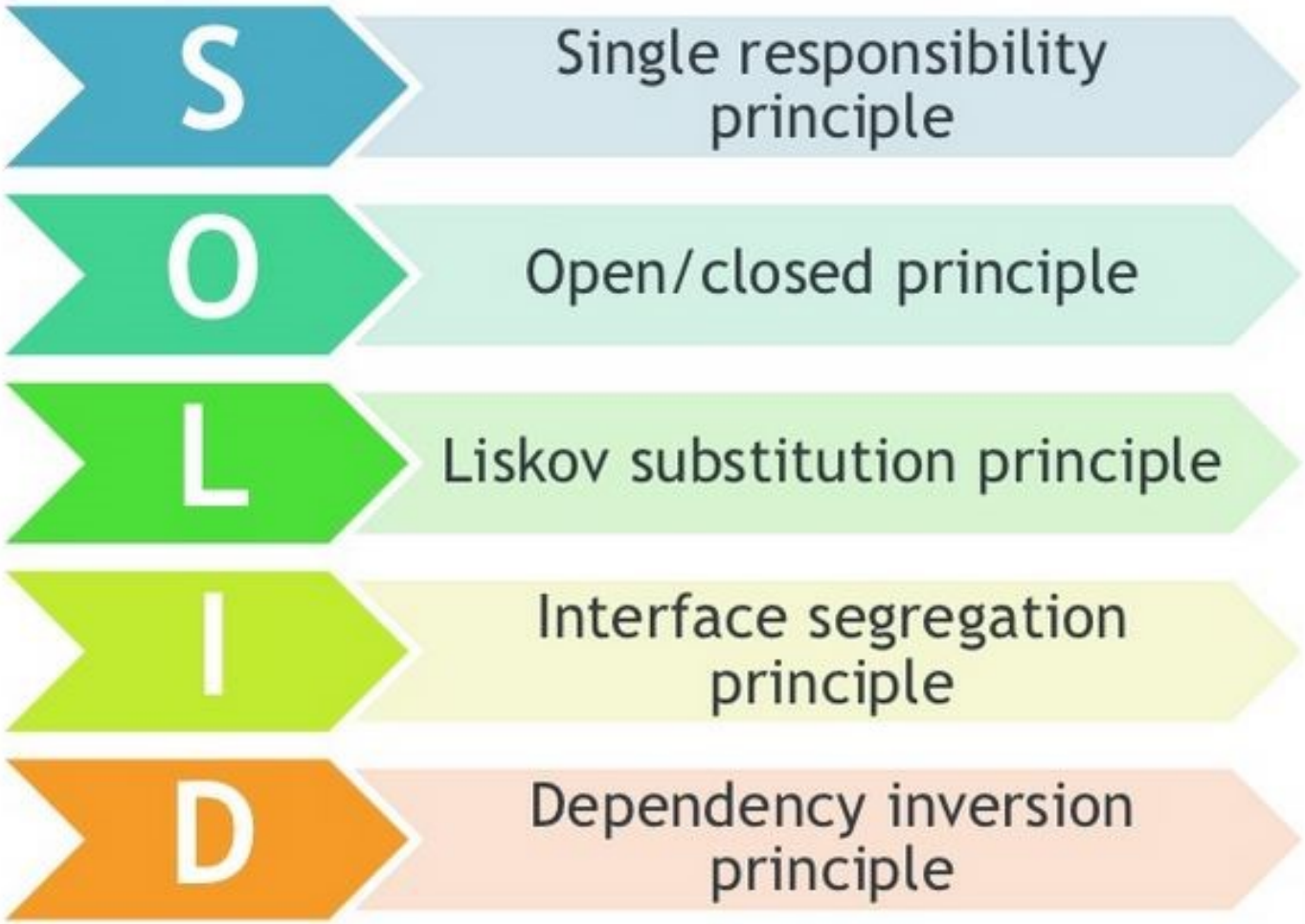
Parametrized tests

http://xunitpatterns.com/Test%20Smells.html

SOLID

In OO computer programming, SOLID is a mnemonic acronym for five design principles intended to make software designs more understandable, flexible and maintainable.

The principles are a subset of many principles promoted by Robert C. Martin in his 2000 paper Design Principles and Design Patterns, SOLID acronym was introduced later by Michael Feathers.



Single Responsibility Principle

”A class should have only one reason to change.”

Violation:

The class has too many responsibilities. Interactions between responsibilities are buried within the class. You design yourself into a corner where it is difficult to alter one responsibility at a time. Tests do not have a clear border between these interactions.

Warning signs:

- Summing up what the class does includes the word “and”
- Class would be challenging for new team members to read and quickly “get it”
- Class has fields that are only used in some methods
- Class has static methods that only operate on parameters

Example

How is sending email related to an interface created to get an employee from a store and add an employee to a store?

```
public interface IEmployeeStore
{
    public Employee getEmployeeById(Long id);

    public void addEmployee(Employee employee);

    public void sendEmail(Employee employee, String content);
}
```

IntelliJ – Refactor > Extract > Interface

```
public interface IEmployeeStore
{
    public Employee getEmployeeById(Long id);

    public void addEmployee(Employee employee);
}
```

```
public interface IEmailSender
{
    public void sendEmail(Employee employee, IEmailContent content
}
```

https://howtodoinjava.com/design-patterns/single-responsibility-principle/

Open/closed Principle

"Software components should be open for extension, but closed for modification"

Violation:

In order to modify the flow of control in specific conditions, you must change the code of the class. You must change the class code whenever a new requirement arrives. Any change may introduce a regression - things that were working before are now broken. Also you must modify the unit tests.

Warning signs:

- Class is doing too much (complementary with SRP)
- Code containing type or enum switch and/or if statements
- Multiple methods with a single parameter coming from single hierarchy

Example

If a new operation is added, do I have to modify my class?

```java
public class SimpleCalculator implements ICalculator
{
    @Override
    public void calculate(IOperation operation)
    {
        if(operation == null) {
            throw new InvalidParameterException("Some message");
        }

        if(operation instanceof Addition) {
            Addition obj = (Addition) operation;
            obj.setResult(obj.getFirstOperand() + obj.getSecondOperand());
        } else if(operation instanceof Substraction) {
            Addition obj = (Addition) operation;
            obj.setResult(obj.getFirstOperand() - obj.getSecondOperand());
        }
    }
}
```

```java
public class SimpleCalculator implements ICalculator
{
    @Override
    public void calculate(IOperation operation)
    {
        if(operation == null) {
            throw new InvalidParameterException("Some message");
        }

        operation.performOperation();
    }
}
```

https://howtodoinjava.com/design-patterns/open-closed-principle/

Liskov Substitution Principle

“Derived types must be completely substitutable for their base types.”

Violation:

Child class inherited from base class is modifying the expected behavior.

Warning signs:

- Class in hierarchy with methods unrelated to its functionality.
- Something feels odd about the class.
- The class or its methods seems to be misplaced.
- Run-time type identification cast to appropriate reference type.

Example

Square has methods to set width and height even though it is not how square works.

```
class Rectangle
{
    protected int m_width;
    protected int m_height;

    public void setWidth(int width){
        m_width = width;
    }

    public void setHeight(int height){
        m_height = height;
    }


    public int getWidth(){
        return m_width;
    }

    public int getHeight(){
        return m_height;
    }

    public int getArea(){
        return m_width * m_height;
    }
}
```

```
class Square extends Rectangle
{
    public void setWidth(int width){
        m_width = width;
        m_height = width;
    }

    public void setHeight(int height){
        m_width = height;
        m_height = height;
    }

}
```

Introduce new parent called Shape

```
private static Rectangle getNewRectangle()
{
    return new Square();
}

public static void main (String args[])
{
    Rectangle r = LspTest.getNewRectangle();

    r.setWidth(5);
    r.setHeight(10);

    System.out.println(r.getArea());
}
```

50 is expected, but got 100

https://www.oodesign.com/liskov-s-substitution-principle.html

Interface Segregation Principle

"Clients should not be forced to implement unnecessary methods which they will not use"

Violation:

Interface declares more than it is really needed (SRP for interfaces)

Warning signs:

- Summing up what the interface does includes the word "and"
- Client must implement method that are not really needed and may not have capabilities to so.
- Interface methods implemented as blank or throwing NotImplementedException.

```
interface IReportable
{
    Stream generateExcel();

    Stream generatePdf();
}
```

Dependency Inversion Principle

“Depend on abstractions, not on concretions”

Violation:

Use of the new operator.

Static methods/classes/singletons.

Warning signs:

- In order to support another type from the same hierarchy, new method overload must be added (SRP).
- It is difficult to create object trees with complex dependencies.
- In order to swap dependency, code must be modified (OCP)

Inversion of Control

IoC is when you have someone else create objects for you.

The control of the object creation is inverted, the programmer no longer has that power over some objects. Instead of calling `new MyClass` something else (*IoC container*) is called to create the instance.

This simple explanation illustrates some very important ideas:

- It is called IoC because control of the object is inverted.
- It is not the programmer, but something else (DI container) that controls the objects.
- IoC is relative – it only applies to some objects of the application. While IoC can be used for some objects, others will remain under the direct control of the programmer.

Dependency Injection

Having DI is the opposite to having hardcoded dependencies.

Dependency injection (DI) is a pattern through which to implement IoC, where the control being inverted is the setting of object's dependencies. It is one of the mechanisms how to achieve IoC, the others are Strategy design pattern, Service Locator pattern, and Factory pattern.

DI is a technique whereby one object (or static method) supplies the dependencies of another object. A dependency is an object that can be used (a service). An injection is the passing of a dependency to a dependent object (a client) that would use it. The service is made part of the client's state. Passing the service to the client, rather than allowing a client to build or find the service, is the fundamental requirement of the pattern.

This fundamental requirement means that using values (services) produced within the class from **new or static methods is prohibited**. The client should accept values passed in from outside. This allows the client to make acquiring dependencies someone else's problem.

Dependency Injection has become one of the cornerstones of modern software engineering, as it is **fundamental to allow proper testing**.

IoC, DI & why

Class `MyClass` has a hardcoded dependency on class `EmailService`. This hidden dependency is not obvious since it is resolved internally somewhere inside the class and the developer has no knowledge of it.

Well, by looking at the code, you can tell there is the dependency, but what if the source is not available. In this code class `MyClass` is coupled to class `EmailService`.

```
public class MyClass
{
    private EmailService emailService = new EmailService();

    public void action()
    {
        System.out.println('MyClass with hidden dependency');
        emailService.sendEmail();
    }
}
```

IoC, DI & why

Class has again dependency on class `EmailService` but this time the dependency is explicit and new instance of the class cannot be created without explicitly providing the dependency value.

The provided value might be a null reference, but that depends on the case, otherwise the provided dependency value should be checked.

Constructor injection

```
public class MyClass
{
    private final EmailService _emailService; // readonly field holding the dependency value

    public MyClass(EmailService emailService) // dependency value is injected through constructor
    {
        if (emailService == null) // null is not allowed as a dependency value!
        {
            throw new IllegalArgumentException("emailService");
        }
        _emailService = emailService;
    }

    public void action()
    {
        System.out.println('MyClass with explicit dependency');
        _emailService.sendEmail();
    }
}
```

Better solution but still violating ISP, using interface would be much better!

```
public void main(String[] args)
{
    MyClass myClass = new MyClass(new RealEmailService());
    myClass.action();

    myClass = new MyClass(new DummyEmailService());
    myClass.action();
}
```

Dependency can be easily replaced if it has the same contract (OCP)

Dependency Entanglement

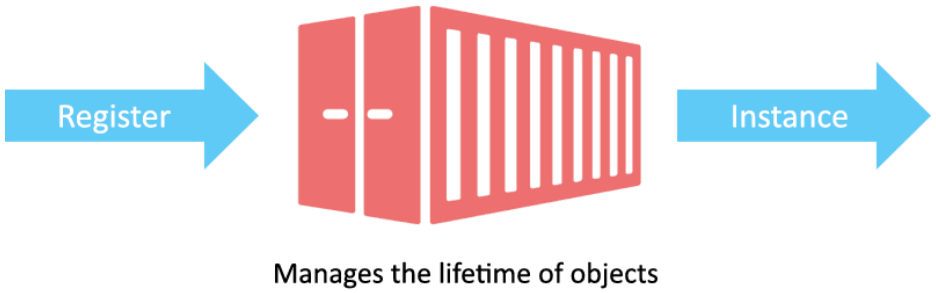
In real world applications the number of dependencies can be high, and the dependencies can also depend on other dependencies with the dependency tree having many levels. Handling this entanglement manually can cause a headache to the developer.

And with the object lifetimes this will become unmaintainable quite quickly.

The process of adding another dependency to an existing class will become very difficult – just adding the constructor parameter is trivial, but then the developer must change (refactor) all the call sites invoking this constructor.

```
public class Main
{
    public static void main(String[] args)
    {
        //All dependencies need to be managed by the developer
        MyClass3 myClass3 = new MyClass3();
        MyClass2 myClass2 = new MyClass2(myClass3);
        MyClass1 myClass1 = new MyClass1(myClass2);
        // myClass1.doSomething();
    }
}

public class MyClass3
{
}

//MyClass2 depends on MyClass3
public class MyClass2
{
    private MyClass3 _myClass3;

    public MyClass2(MyClass3 myClass3)
    {
        _myClass3 = myClass3;
    }
}

// MyClass1 depends on MyClass2
public class MyClass1
{
    private MyClass2 _myClass2;

    public MyClass1(MyClass2 myClass2)
    {
        _myClass2 = myClass2;
    }
}
```

DI Containers



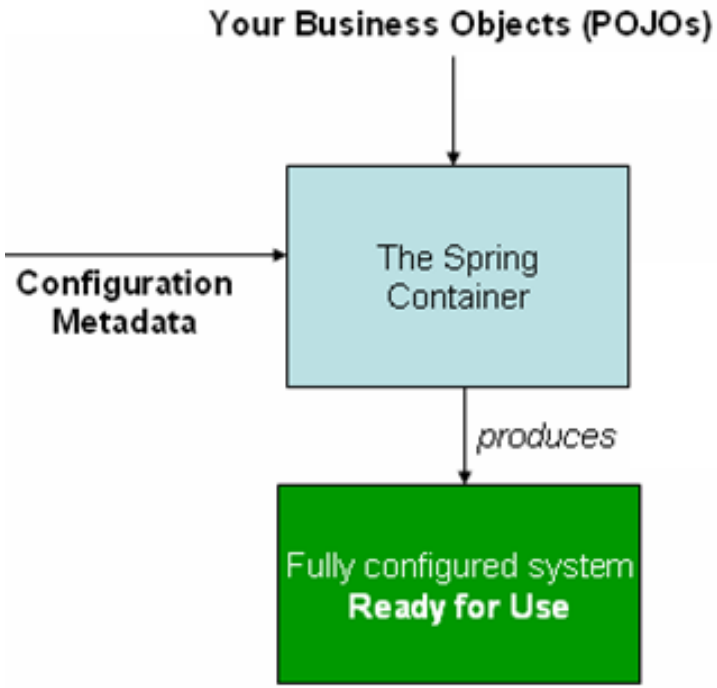
Register: The container must know which dependency to instantiate when it encounters a particular type. This process is called registration – type-mapping.

Resolve: When using the IoC container, objects are not created manually. The container does it for the developer. This is called resolution. The container must include some methods to resolve the specified type; the container creates an object of the specified type, injects the required dependencies if any and returns the object.

Dispose: The container must manage the lifetime of the dependent objects. Most containers include different lifetime managers to manage an object's lifecycle and dispose it.

This is how a singleton can be defined.

Spring DI Container




```java
@Configuration
public class ServiceConfig {

    @Bean
    public TransferService transferService(AccountRepository accountRepository) {
        return new TransferServiceImpl(accountRepository);
    }
}
```

```java
public static void main(String[] args) {
    ApplicationContext ctx = new AnnotationConfigApplicationContext(SystemTestConfig.class);
    // everything wires up across configuration classes...
    TransferService transferService = ctx.getBean(TransferService.class);
    transferService.transfer(100.00, "A123", "C456");
}
```

Test Doubles

A test double is an object that can stand in for a real object in a test, like a stunt double stands in for an actor in a movie.

Test double is an object which we use in place of a real object during a test

Types of test doubles:

- Dummy – just a value to make the compiler happy
- Fake – working & functional unit with shortcut making it helpful in testing, like an in-memory database
- Stub – object with hardcoded canned responses (state verification)
- Spy – “records” values during the executions of the test
- Mock – object used for verification that a certain method was called with specific parameters (behavior verification)

http://xunitpatterns.com/Test%20Double.html

Test Doubles

The purpose of test doubles is

- to isolate the code that is being tested.
- and to define (hard-code) the behavior of the dependencies so it suits the needs of the test
  - Do you want to verify that the code does fail gracefully?
    → Setup a stub so it will throw an exception when certain method is called.
  - Do you want to verify that certain method was called exactly 3 times during the test execution?
    → Set up a mock for the call of the method with expected parameters and verify it afterwards.
  - Do you want to simulate a response from a SMTP server?
    → Setup a stub for the SMTP service in your code so it will return the required response.

Mockito

https://site.mockito.org/

Popular Java library for mocking and verification.

```
@Test
public void whenNotUseMockAnnotation_thenCorrect() {
    List mockList = Mockito.mock(ArrayList.class);

    mockList.add("one");
    Mockito.verify(mockList).add("one");
    assertEquals(0, mockList.size());

    Mockito.when(mockList.size()).thenReturn(100);
    assertEquals(100, mockList.size());
}
```

mockList "is" ArrayList

Size returns 100, even though the list "contains" 1 item!

```
@Test
public void whenAddCalledVerified() {
    MyList myList = mock(MyList.class);
    doNothing().when(myList).add(isA(Integer.class), isA(String.class));
    myList.add(0, "");

    verify(myList, times(1)).add(0, "");
}
```

Verify that method 'add' was called once with these parameters

https://www.baeldung.com/mockito-series

AssertJ

https://assertj.github.io/doc/

AssertJ is a java library providing a rich set of assertions, truly helpful error messages, improves test code readability and is designed to be super easy to use within IDE.

Fluent = method can be chained

```
assertThat(frodo)
  .isNotEqualTo(sauron)
  .isIn(fellowshipOfTheRing);

assertThat(frodo.getName())
  .startsWith("Fro")
  .endsWith("do")
  .isEqualToIgnoringCase("frodo");

assertThat(fellowshipOfTheRing)
  .hasSize(9)
  .contains(frodo, sam)
  .doesNotContain(sauron);
```

Assertions are easier to read

```
assertEquals(expectedSize, myList.size())
vs
assertThat(myList).hasSize(expectedSize)
```

https://www.baeldung.com/introduction-to-assertj

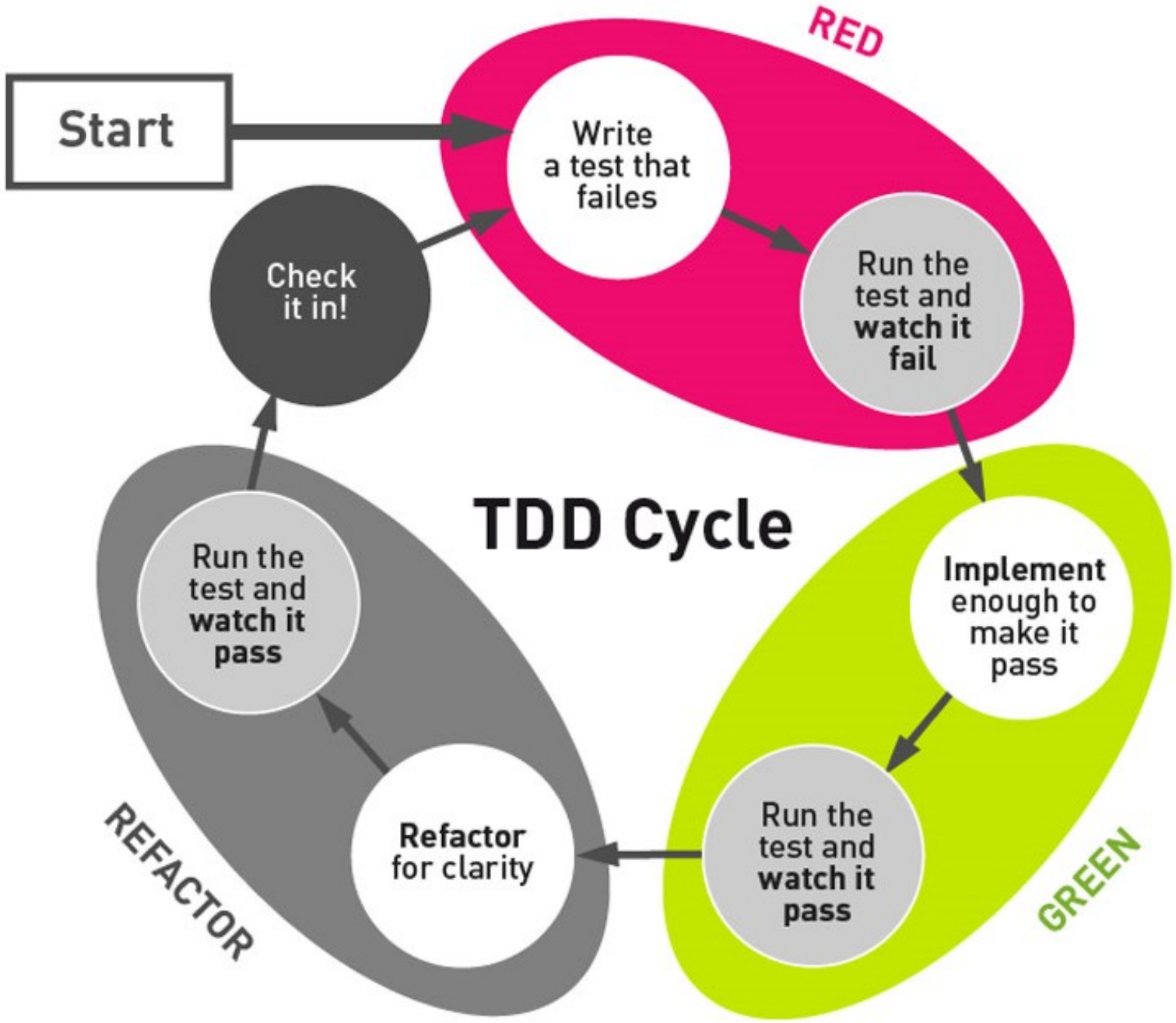
```java
@ParameterizedTest
@ValueSource(ints = {1, 3, 5, -3, 15, Integer.MAX_VALUE})
void isOdd_ShouldReturnTrueForOddNumbers(int number) {
    assertTrue(Numbers.isOdd(number));
}

@ParameterizedTest
@CsvFileSource(resources = "/data.csv", numLinesToSkip = 1)
void toUpperCase_ShouldGenerateTheExpectedUppercaseValueCSVFile(String input,
    String actualValue = input.toUpperCase();
    assertEquals(expected, actualValue);
}
```

- ParameterizedTests [Runner: JUnit 5] (0
  - test_ValueSource_String(String) (0.01
    - [1] 1 (0.011 s)
    - [2] 2 (0.001 s)
    - [3] 3 (0.004 s)
  - test_ValueSource_Objects(Book) (0.0
    - [1] Harry Potter (0.005 s)
    - [2] Hamlet (0.005 s)
  - test_MethodSource_Objects(Book) (0
  - test_EnumSource(ElementType) (0.00
  - test_MethodSource_MultipleParams(i
  - test_MethodSource(String) (0.002 s)
    - [1] A (0.002 s)
    - [2] B (0.003 s)
  - test_ValueSource(int) (0.001 s)
  - test_CsvFileSource(String, int) (0.001
  - test_EnumSource_Extended(ElementT
    - [1] TYPE (0.002 s)
    - [2] FIELD (0.001 s)
    - [3] METHOD (0.006 s)
  - test_CsvSource(int, String) (0.001 s)

Parametrized Test

Test Driven Development

TDD is a SW development process that relies on the repetition of a very short development cycle: requirements are turned into very specific test cases, then the software is improved so that the tests pass. This is opposed to software development that allows software to be added that is not proven to meet requirements.



Other things to know

Design patterns

DRY – don’t repeat yourself

- Don’t reinvent the wheel, use abstraction and design patterns

YAGNI – you aren’t gonna need it

- Don’t add a new functionality until necessary

KISS – keep it simple, stupid

- most systems work best if they are kept simple rather than made complicated

If it ain't broke, don't fix it

- Don’t try to change things that are not broken (be sure you have thoroughly test it if you decide to fix it)

Further reading

https://lostechies.com/derekgreer/2011/03/07/effective-tests-introduction/

http://misko.hevery.com/attachments/Guide-Writing%20Testable%20Code.pdf

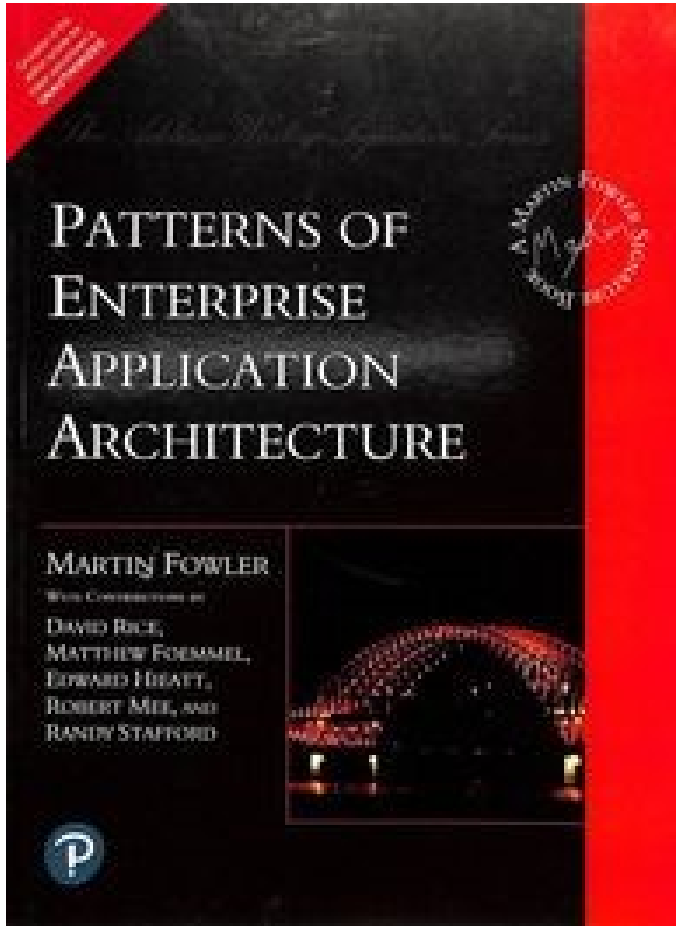
https://www.slideshare.net/stevepgreen/are-you-a-solid-coder

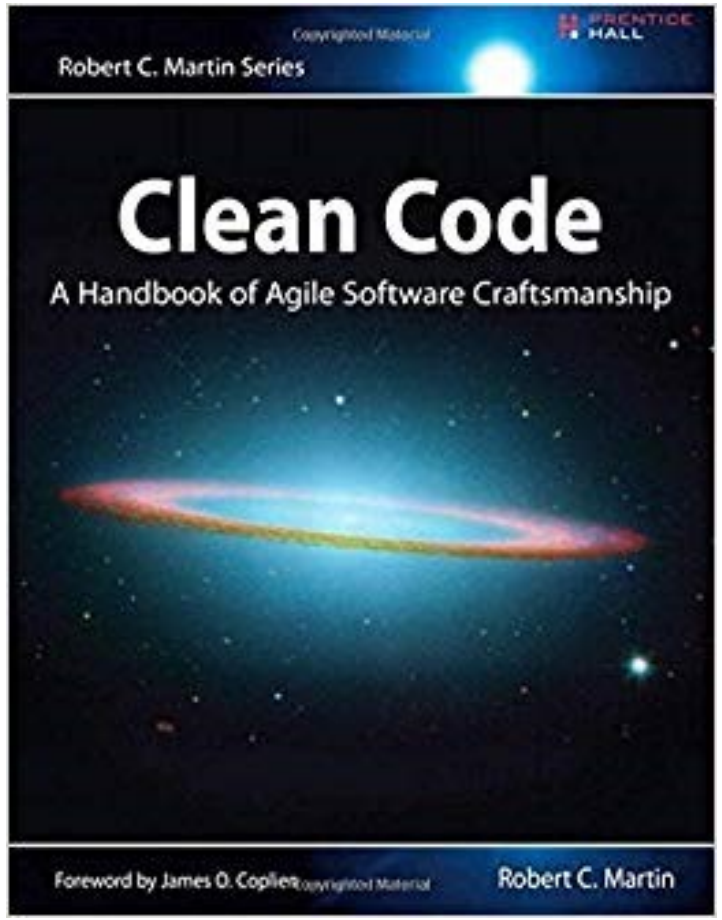
https://blog.cleancoder.com/

https://medium.com/@fagnerbrack/code-comment-is-a-smell-4e8d78b0415b

https://martinfowler.com/articles/mocksArentStubs.html

Books


PATTERNS OF
ENTERPRISE
APPLICATION
ARCHITECTURE
MARTIN FOWLER
DAVID RICE,
MATTHEW FOEMMEL,
EDWARD HIEATT,
ROBERT MEE, AND
RANDY STAFFORD


Robert C. Martin Series
Clean Code
A Handbook of Agile Software Craftsmanship
Foreword by James O. Coplien
Robert C. Martin

CARL QUIT. HE'S THE ONLY ONE WHO KNOWS HOW TO PROGRAM THE LEGACY SYSTEM.
IT CAN'T BE THAT HARD. GO FIGURE IT OUT.
?
FRACK.
www.dilbert.com scottadams@aol.com
12-9-06 ©2006 Scott Adams, Inc./Dist. by UFS, Inc.

Thank you

Junit 5 Jupiter

JUnit 5 = JUnit Platform + JUnit Jupiter + JUnit Vintage

FEATURE	JUNIT 4	JUNIT 5
Declare a test method	`@Test`	`@Test`
Execute before all test methods in the current class	`@BeforeClass`	`@BeforeAll`
Execute after all test methods in the current class	`@AfterClass`	`@AfterAll`
Execute before each test method	`@Before`	`@BeforeEach`
Execute after each test method	`@After`	`@AfterEach`
Disable a test method / class	`@Ignore`	`@Disabled`
Test factory for dynamic tests	NA	`@TestFactory`
Nested tests	NA	`@Nested`
Tagging and filtering	`@Category`	`@Tag`
Register custom extensions	NA	`@ExtendWith`

Extensibility

```java
@ExtendWith(TimingExtension.class)
class TimingExtensionTests {

    @Test
    void sleep20ms() throws Exception {
        Thread.sleep(20);
    }

    @Test
    void sleep50ms() throws Exception {
        Thread.sleep(50);
    }

}
```

```
INFO: Method [sleep20ms] took 24 ms.
INFO: Method [sleep50ms] took 53 ms.
```

```java
public class TimingExtension implements BeforeTestExecutionCallback, AfterTestExecutionCallback
{

    private static final Logger logger = Logger.getLogger(TimingExtension.class.getName());

    private static final String START_TIME = "start time";

    @Override
    public void beforeTestExecution(ExtensionContext context) throws Exception {
        getStore(context).put(START_TIME, System.currentTimeMillis());
    }

    @Override
    public void afterTestExecution(ExtensionContext context) throws Exception {
        Method testMethod = context.getRequiredTestMethod();
        long startTime = getStore(context).remove(START_TIME, long.class);
        long duration = System.currentTimeMillis() - startTime;

        logger.info(() ->
            String.format("Method [%s] took %s ms.", testMethod.getName(), duration));
    }

    private Store getStore(ExtensionContext context) {
        return context.getStore(Namespace.create(getClass(), context.getRequiredTestMethod()));
    }

}
```